Dossier Técnico
Arquitectura de Seguridad en Enrutamiento con Angular (Functional Guards & RBAC)

- **Módulo Profesional:** Desarrollo de Interfaces (Código: 0488) — 2º Curso DAM.
- **Alineación Curricular (Comunidad de Madrid):**
  - **RA 1:** Confecciona interfaces de usuario optimizando la experiencia de usuario y aplicando patrones de diseño arquitectónicos.
    - **CE 1.b:** Se han identificado y aplicado patrones de diseño arquitectónicos adecuados (MVVM, separación estricta de responsabilidades).
    - **CE 1.c:** Se ha implementado un sistema de enrutamiento dinámico, modular y desacoplado mediante carga diferida (*lazy loading*).
    - **CE 1.d:** Se han establecido mecanismos de control de acceso, protección de rutas y gestión de sesiones basados en tokens/roles.
    - **CE 1.e:** Se ha garantizado la persistencia, integridad y reactividad del estado de autenticación.

1. Base Teórica Técnica: Paradigma Funcional y Enrutamiento Defensivo

Históricamente, Angular implementaba el control de acceso en el router a través de interfaces (CanActivate, CanActivateChild, CanMatch, CanDeactivate) implementadas en clases @Injectable(). En las versiones modernas de Angular, las interfaces basadas en clases han sido completamente deprecadas y reemplazadas por **Functional Router Guards** (CanActivateFn, CanMatchFn, etc.).

1.1. Ventajas Arquitectónicas del Enfoque Funcional

- **Optimización en Bundling (*Tree-Shaking*):** Las funciones puras son directamente evaluables por herramientas de empaquetado como esbuild o Rollup, eliminando metadatos innecesarios de clases e inyectores complejos en tiempo de compilación.
- **Composabilidad y Factorías:** Permite generar guardas de orden superior (*higher-order functions*) parametrizables, evitando duplicar clases para validar roles o contextos distintos.
- **Inyección Contextual en Ejecución:** Empleo de la función inject() de @angular/core para resolver dependencias (Router, AuthService) dentro del contexto de inyección del enrutador sin constructores intermedios.

1.2. Estratificación Defensiva: CanMatchFn vs. CanActivateFn

- **CanMatchFn (Primera Línea - Nivel Red):** Determina si una ruta puede coincidir y, fundamentalmente, si su fragmento de código (*chunk* de JavaScript diferido) debe descargarse por red. Si retorna false o redirige, el navegador jamás realiza la petición HTTP para descargar el archivo JS compilado del módulo perezoso. Esto impide la inspección de código propietario o interfaces de administración por parte de usuarios no autenticados o con roles inferiores.
- **CanActivateFn (Segunda Línea - Nivel Vista):** Se evalúa una vez resuelta la ruta y descargado el código fuente, validando si el componente asociado tiene permiso para ser instanciado y renderizado en el <router-outlet>.

1.3. Manejo Atómico de Navegación: UrlTree vs. Primitivo Booleano

Retornar un valor booleano false cancela la navegación silenciosamente, congelando al usuario en la vista actual sin retroalimentación, o requiriendo llamadas imperativas manuales a router.navigate(). Esta práctica es propensa a condiciones de carrera (*race conditions*) en el stack de navegación. El estándar técnico exige retornar un **UrlTree** (generado con Router.createUrlTree()), el cual instruye al enrutador a abortar la navegación original y reencaminar el flujo atómicamente en un único ciclo de resolución de rutas.

2. Aplicación Práctica

2.1. Capa de Dominio y Estado Reactivo: auth.service.ts

El servicio centraliza la sesión del usuario empleando **Angular Signals**, proporcionando granularidad reactiva y cómputos derivados libres de subscripciones manuales.

```
TypeScript
import { Injectable, computed, signal } from '@angular/core';

export type UserRole = 'ADMIN' | 'STANDARD_USER';

export interface UserSession {
  readonly id: string;
  readonly username: string;
  readonly roles: readonly UserRole[];
  readonly token: string;
}

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private readonly STORAGE_KEY = 'app_session_v1';

  // Estado reactivo privado mediante Signals
  private readonly _currentSession = signal<UserSession | null>(this.restoreSession());

  // Selectores públicos computados (solo lectura)
  public readonly session = this._currentSession.asReadonly();
  public readonly isAuthenticated = computed(() => this._currentSession() !== null);
  public readonly roles = computed(() => this._currentSession()?.roles ?? []);

  public login(sessionData: UserSession): void {
    localStorage.setItem(this.STORAGE_KEY, JSON.stringify(sessionData));
    this._currentSession.set(sessionData);
  }

  public logout(): void {
    localStorage.removeItem(this.STORAGE_KEY);
    this._currentSession.set(null);
  }

  public hasRole(role: UserRole): boolean {
```

```
    return this.roles().includes(role);
  }

  public hasAnyRole(requiredRoles: readonly UserRole[]): boolean {
    const currentRoles = this.roles();
    return requiredRoles.some(role => currentRoles.includes(role));
  }

  private restoreSession(): UserSession | null {
    const raw = localStorage.getItem(this.STORAGE_KEY);
    if (!raw) return null;
    try {
      return JSON.parse(raw) as UserSession;
    } catch {
      localStorage.removeItem(this.STORAGE_KEY);
      return null;
    }
  }
}
```

2.2. Guarda de Autenticación con Preservación de Estado: auth.guard.ts

Intercepta solicitudes no autenticadas, capturando la URL intentada para permitir redirección posterior una vez completado el *login*.

```
TypeScript
import { inject } from '@angular/core';
import { CanActivateFn, Router, ActivatedRouteSnapshot, RouterStateSnapshot } from
'@angular/router';
import { AuthService } from './auth.service';

export const authGuard: CanActivateFn = (
 route: ActivatedRouteSnapshot,
 state: RouterStateSnapshot
) => {
 const authService = inject(AuthService);
 const router = inject(Router);

 if (authService.isAuthenticated()) {
  return true;
 }

 // Redirección atómica con persistencia del parámetro returnUrl
 return router.createUrlTree(['/login'], {
  queryParams: { returnUrl: state.url }
 });
};
```

2.3. Factoría de Autorización Parametrizada (RBAC): role.guard.ts

Implementación de factorías de orden superior que generan instancias de CanMatchFn y CanActivateFn para control de acceso basado en roles (*Role-Based Access Control*).

TypeScript

```
import { inject } from '@angular/core';
import { CanActivateFn, CanMatchFn, Route, UrlSegment, Router } from '@angular/router';
import { AuthService, UserRole } from './auth.service';

/**
 * Guarda a nivel CanMatch: Bloquea la descarga de módulos perezosos a usuarios no autorizados.
 */
export const roleMatchGuard = (allowedRoles: readonly UserRole[]): CanMatchFn => {
  return (route: Route, segments: UrlSegment[]) => {
    const authService = inject(AuthService);
    const router = inject(Router);

    if (!authService.isAuthenticated()) {
      return router.createUrlTree(['/login']);
    }

    if (authService.hasAnyRole(allowedRoles)) {
      return true;
    }

    return router.createUrlTree(['/forbidden']);
  };
};

/**
 * Guarda a nivel CanActivate: Protege la activación del componente ya descargado.
 */
export const roleActivateGuard = (allowedRoles: readonly UserRole[]): CanActivateFn => {
  return () => {
    const authService = inject(AuthService);
    const router = inject(Router);

    if (!authService.isAuthenticated()) {
      return router.createUrlTree(['/login']);
    }

    if (authService.hasAnyRole(allowedRoles)) {
```

```
    return true;
   }

   return router.createUrlTree(['/forbidden']);
  };
};
```

2.4. Guarda de Integridad de Datos en Vistas Reactivas: pending-changes.guard.ts

Protege contra la pérdida involuntaria de información mediante CanDeactivateFn, vinculándose contractualmente a los componentes mediante una interfaz desacoplada.

TypeScript

```
import { CanDeactivateFn } from '@angular/router';

export interface ComponentCanDeactivate {
  canDeactivate: () => boolean | Promise<boolean>;
}

export const pendingChangesGuard: CanDeactivateFn<ComponentCanDeactivate> =
(component) => {
  // Si el componente implementa la interfaz, se evalúa su estado interno (p. ej. form.dirty)
  if (component.canDeactivate) {
    return component.canDeactivate();
  }
  return true;
};
```

Ejemplo de implementación en el componente con formulario reactivo:

TypeScript

```
import { Component } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { ComponentCanDeactivate } from './pending-changes.guard';

@Component({
  selector: 'app-user-form',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="userForm">
      <input formControlName="name" placeholder="Nombre completo" />
    </form>
  `
})
export class UserFormComponent implements ComponentCanDeactivate {
  public readonly userForm = new FormGroup({
    name: new FormControl('', { nonNullable: true, validators: [Validators.required] })
  });

  public canDeactivate(): boolean {
    if (this.userForm.dirty) {
      return window.confirm('Existen cambios sin guardar en el formulario. ¿Desea descartar la información?');
    }
    return true;
  }
}
```

2.5. Ensamblado del Enrutador: app.routes.ts

Definición de rutas integrando las protecciones en múltiples niveles defensivos (*Defense in Depth*).

```
TypeScript
import { Routes } from '@angular/router';
import { authGuard } from './auth.guard';
import { roleActivateGuard, roleMatchGuard } from './role.guard';
import { pendingChangesGuard } from './pending-changes.guard';

export const routes: Routes = [
 {
  path: '',
  pathMatch: 'full',
  redirectTo: 'login'
 },
 {
  path: 'login',
  loadComponent: () => import('./features/auth/login.component').then(m =>
m.LoginComponent)
 },
 {
  path: 'forbidden',
  loadComponent: () => import('./core/pages/forbidden.component').then(m =>
m.ForbiddenComponent)
 },
 // Zona de usuario estándar y administración (Autenticación requerida)
 {
  path: 'workspace',
  canActivate: [authGuard],
  children: [
   {
    path: '',
    loadComponent: () => import('./features/dashboard/dashboard.component').then(m =>
m.DashboardComponent)
   },
   {
    path: 'editor',
    canDeactivate: [pendingChangesGuard],
    loadComponent: () => import('./features/editor/editor.component').then(m =>
m.UserFormComponent)
   }
```

```
    ]
  },
  // Zona exclusiva de administración: Blindaje de Red (canMatch) y Activación (canActivate)
  {
    path: 'admin',
    canMatch: [roleMatchGuard(['ADMIN'])],
    canActivate: [roleActivateGuard(['ADMIN'])],
    loadChildren: () => import('./features/admin/admin.routes').then(m => m.ADMIN_ROUTES)
  },
  {
    path: '**',
    loadComponent: () => import('./core/pages/not-found.component').then(m =>
m.NotFoundComponent)
  }
];
```

Functional Router Guards (`CanActivateFn`, `CanMatchFn`, `CanDeactivateFn`)

Mecanismos de intercepción de navegación en Angular implementados como funciones puras en lugar de servicios basados en clases. Eliminan el uso del decorador `@Injectable()` y constructores para la inyección de dependencias, permitiendo el acceso directo a servicios mediante la función `inject()`. Facilitan el *tree-shaking* durante la compilación y reducen el tamaño final del *bundle*.

`CanMatchFn`

Tipo de guarda que se evalúa antes de resolver la coincidencia de una ruta y antes de iniciar la descarga de cualquier módulo o componente configurado mediante carga diferida (*lazy loading*). Si retorna `false` o un `UrlTree`, el navegador **no descarga los artefactos JavaScript correspondientes**, actuando como una barrera de seguridad a nivel de red y consumo de ancho de banda.

`CanActivateFn`

Guarda de ejecución posterior a la resolución y descarga de la ruta. Determina si el componente destino puede instanciarse y renderizarse en el `<router-outlet>`. A diferencia de `CanMatch`, evalúa los permisos cuando el código fuente del componente ya está presente en el cliente.

`CanDeactivateFn`

Guarda que intercepta los intentos de navegación saliente desde la ruta activa hacia cualquier otra ubicación. Se utiliza para salvaguardar la integridad de datos volátiles, comprobando si existen cambios sin guardar en formularios reactivos (`form.dirty`) antes de destruir el componente de la vista.

`UrlTree`

Estructura de datos interna e inmutable de Angular que representa una URL desglosada de forma jerárquica (segmentos, parámetros de consulta y fragmentos). En el contexto de los guards funcionales, devolver una instancia de `UrlTree` generada mediante `Router.createUrlTree()` permite abortar la navegación original y redirigir al usuario hacia una nueva ruta de manera atómica, evitando efectos colaterales o llamadas manuales a `Router.navigate()`.

RBAC (*Role-Based Access Control*)

Modelo de control de acceso en el que los permisos y privilegios de navegación se asocian a roles definidos (por ejemplo, `ADMIN` o `STANDARD_USER`) en lugar de asignarse individualmente a cada usuario. En el cliente, determina qué vistas y submódulos están disponibles según el perfil extraído de la sesión o del token JWT.

Factoría de Guards (*Higher-Order Functions*)
Patrón de diseño arquitectónico donde una función recibe parámetros de configuración (como una lista de roles permitidos o una URL de desvío) y retorna una función guarda (`CanActivateFn` o `CanMatchFn`). Permite parametrizar y reutilizar la lógica de autorización en múltiples definiciones de rutas sin duplicar código.

Inyección Contextual (`inject()`)
Función introducida en las versiones modernas de Angular que permite resolver dependencias del inyector activo dentro de un contexto de inyección válido (como la ejecución de un guard funcional, un interceptor HTTP o el inicializador de una propiedad de clase) sin depender de la inyección por constructor.

Signals (`signal()`, `computed()`)
Primitivas de reactividad granular que gestionan el estado de forma síncrona y predecible. En la arquitectura de autenticación:

- `signal`: Almacena el estado mutable de la sesión actual de forma encapsulada.
-
- `computed`: Define valores derivados de solo lectura (como `isAuthenticated` o `roles`) que se recalculan automáticamente solo cuando la señal origen cambia de valor.
-

Carga Diferida (*Lazy Loading*)
Técnica de optimización que divide la aplicación en múltiples fragmentos (*chunks*) de código JavaScript compilados independientemente. Estos fragmentos solo se solicitan y descargan a través de la red cuando el usuario navega explícitamente a la ruta asociada (mediante `loadChildren` o `loadComponent`), reduciendo drásticamente el tiempo de carga inicial (*First Contentful Paint*).

`returnUrl` (Preservación de Estado de Navegación)
Patrón de experiencia de usuario (UX) donde la URL a la que el usuario intentaba acceder originalmente se almacena temporalmente como parámetro de consulta (*query param*) al redirigirlo a la pantalla de inicio de sesión (`/login?returnUrl=/workspace`). Una vez autenticado, el sistema consume este parámetro para restaurar el flujo original de trabajo.